

The Computing Stack

How a computer gets from electricity to the Python program you wrote. This page connects Weeks 2a and 2b into one picture and sets up our next topic: programming languages.

Week 2a

Introduction to CS

Week 2b

How computers work

Week 3

The big picture (this page)

Next in class

Types of languages

About 25 minutes to read, plus the review game at the end.

START HERE

One idea holds it all together

In Week 2a we said computer science is not really the study of computers, the same way chemistry is not the study of beakers. It is the study of **computation**: how problems get solved step by step, and how we build machines to do the steps for us.

Those machines are complicated. A laptop chip has billions of parts. Nobody, not even the engineers who design it, keeps all of that in their head at once. Instead, computers are built in **layers**. Each layer uses the one below it without needing to know how it works inside.

Hiding the details of a lower layer so you can work with a simpler idea on top is called **abstraction**. You already do this every day. You steer a car without thinking about fuel injection. You text a friend without thinking about cell towers.

★ CONNECTION

Abstraction comes up all year in CSP. When you write `Circle(200, 200, 50)` in CMU CS Academy, you get a circle without writing any of the code that figures out which pixels to color. Someone else built that layer so you could stand on it.

THE BIG PICTURE

The stack, top to bottom

Here is the whole computer as a stack of layers. The top is where you live: ideas and problems in plain language. The bottom is pure physics: electricity flowing or not flowing. Everything else is built in between.

MORE HUMAN-FRIENDLY ↑

You	Problems and ideas, in natural language
Applications	Web browsers, games, Scratch, CMU CS Academy
Operating system	Windows, macOS, Linux. Runs and manages everything above it
Machine language	The binary instructions a CPU can carry out
Hardware	CPU, memory, storage, GPU, input and output devices
Logic gates	AND, OR, NOT, wired together by the millions
Transistors	Billions of tiny switches. On means 1, off means 0

Programming languages

How people write the software in these layers. A compiler or interpreter translates it down to machine language.

CLOSER TO THE ELECTRICITY ↓

The rest of this page walks up the stack from the bottom, then follows one program all the way down and back.

BOTTOM LAYERS

Switches and gates

At the very bottom, a computer is made of **transistors**: microscopic switches that either let electricity through or block it. On or off. That is why computers use **binary**, a number system with only two digits: 1 for on and 0 for off.

One switch cannot do much. But wire a few transistors together and you get a **logic gate**, a tiny circuit that takes in 1s and 0s and puts out a 1 or 0 based on a rule. The three basic gates are AND, OR, and NOT. Here is the rule for AND: the output is 1 only when both inputs are 1.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Here is the surprising part: combine enough of these simple gates and you can build circuits that add numbers, compare them, and store them. Stack millions of those circuits together and you have a processor.

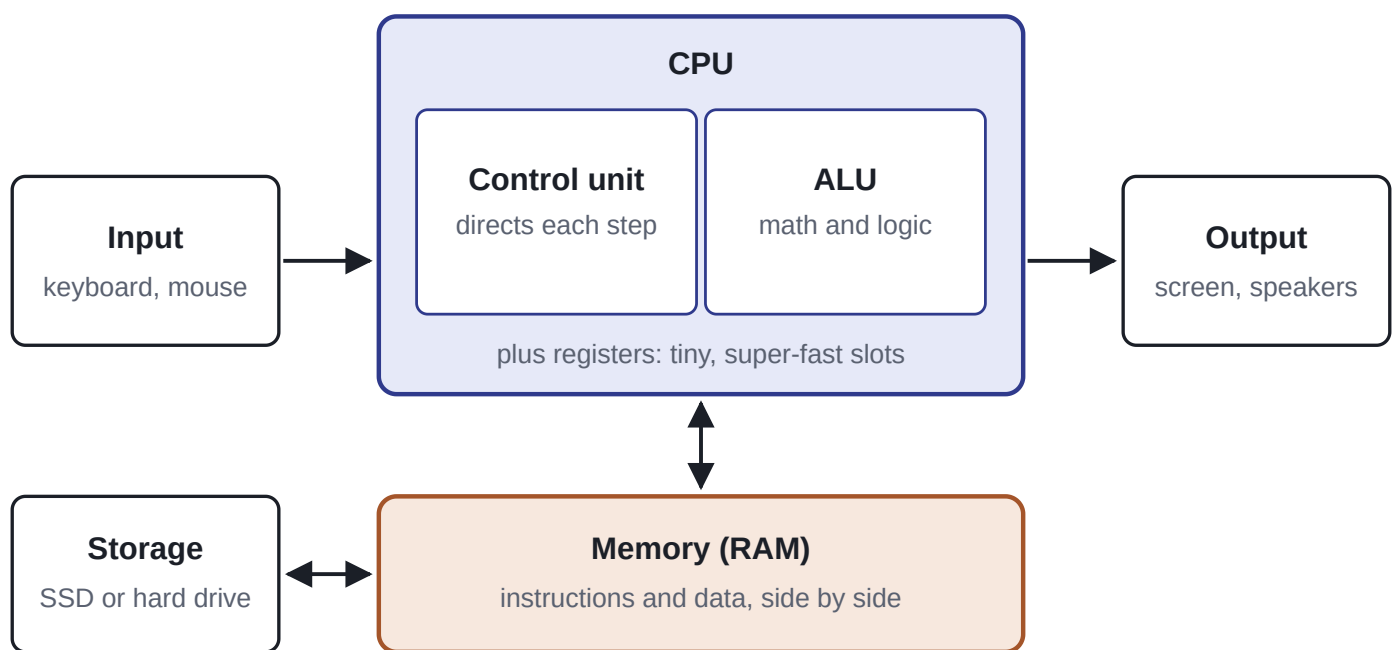
★ CONNECTION

Python has the same three words: `and` , `or` , and `not` . Later this year you will use them to write conditions like `if x > 0 and y > 0:` . They follow the exact same rules as the gates.

HARDWARE LAYER

The Von Neumann architecture

In Week 2b we met the design that nearly every modern computer follows, named for mathematician John von Neumann. Its big idea is that **one memory holds both the program's instructions and its data**. To run a different program, you do not rewire the machine. You just load different instructions into memory.



Every arrow is data moving. The CPU and memory trade information constantly, billions of times a second.

A kitchen, if it helps

PART	KITCHEN VERSION	WHAT IT REALLY DOES
CPU	The cook	Carries out instructions one at a time, very fast
RAM	The countertop	Holds what is in use right now. Fast, but cleared when the power goes off
Storage	The pantry	Keeps programs and files long-term, even with the power off. Slower to reach

PART	KITCHEN VERSION	WHAT IT REALLY DOES
Program	The recipe	The list of instructions, loaded onto the counter before cooking
GPU	A crew of helpers who each do one simple step at the same time	Colors millions of pixels in parallel, so the CPU does not have to
Motherboard	The kitchen itself	The circuit board that connects every part so they can communicate

The fetch-decode-execute cycle

The CPU does one thing over and over: **fetch** the next instruction from memory, **decode** it to work out what it means, and **execute** it. Here is the short assembly program from Week 2b, traced step by step. Memory starts with `ABC = 2` and `XYZ = 3`.

INSTRUCTION FETCHED	DECODED AS	EXECUTED
LOAD REGD, ABC	Copy a value from memory into a register	Register D = 2
LOAD REGE, XYZ	Copy a value from memory into a register	Register E = 3
ADD REGF, REGD, REGE	Ask the ALU to add two registers	Register F = 5
STORE REGF, TOTAL	Copy a register back into memory	TOTAL = 5
HALT	Stop	Done

Five trips around the cycle just to add $2 + 3$. A modern CPU runs billions of cycles per second, which is what a speed like 3 GHz means.

TRY IT

Look up your own computer's hardware. On a Mac, open the Apple menu and choose **About This Mac**. On Windows, go to **Settings** → **System** → **About**. Find the name of the processor (the CPU) and how much memory (RAM) it has.

OPERATING SYSTEM AND APPLICATIONS

Software: system and application

Hardware is the physical stuff you can touch. **Software** is the programs and data that tell the hardware what to do. Week 2b split software into two kinds.

System software runs the computer itself. The biggest piece is the **operating system** (OS). It starts when you turn the computer on and runs until you shut it down. It decides which program gets the CPU next, hands out memory, keeps track of files, and talks to devices through small programs called drivers. Because the OS handles all of that, an app never has to deal with the raw hardware.

Application software is everything you open to get something done: a browser, a word processor, a game, Scratch. Applications ask the OS for what they need, and the OS asks the hardware.

TRY IT

Watch the operating system at work. On a Mac, press **⌘ Space** and type **Activity Monitor**. On Windows, press **Ctrl + Shift + Esc** to open Task Manager. Count roughly how many processes are running. The OS is sharing one CPU and one memory among all of them, right now.

CROSSING THE GAP

The language ladder

The CPU only understands machine language. People think in English, Spanish, Japanese. Human language is full of ambiguity that a computer cannot handle, so we cannot just type English and have it turned into binary. We climb down a ladder instead. Here is the same small job at each rung: add two numbers and save the total.

Natural language
for people

ABC is 2 and XYZ is 3. Add them and save the total.

Pseudocode
two styles; no computer
runs either

PLAIN

Set total to ABC plus XYZ

AP CSP

`total ← abc + xyz`

Plain pseudocode is written however makes sense to you. AP CSP pseudocode follows set rules, like the style on the AP exam reference sheet. Its arrow means “store the value on the right in the variable on the left.”

High-level language

same idea, seven languages

PYTHON `total = abc + xyz`

RUBY `total = abc + xyz`

JAVA `int total = abc + xyz;`

C++ `int total = abc + xyz;`

JAVASCRIPT `let total = abc + xyz;`

GO `total := abc + xyz`

SCRATCH

set `total` to `abc + xyz`

Assembly language

one line per machine instruction

```
LOAD  REGD, ABC
LOAD  REGE, XYZ
ADD   REGF, REGD, REGE
STORE REGF, TOTAL
```

Machine language

binary the CPU runs

OPERATION 4 BITS	REGISTER 4 BITS	LAST 8 BITS	RESULT
0001 LOAD	1101 register D	0000 0001 address 1 (ABC)	D ← 2
0001 LOAD	1110 register E	0000 0010 address 2 (XYZ)	E ← 3
0101 ADD	1111 into register F	1101 1110 registers D and E	F ← D + E = 5
0010 STORE	1111 register F	0000 0000 address 0 (TOTAL)	TOTAL ← 5

From a made-up teaching computer. For LOAD and STORE, the last 8 bits are one memory address. For ADD, they are two 4-bit register numbers, because adding happens inside the CPU. Real CPUs use longer, more complicated codes, but the idea is the same.

Every high-level version says the same thing, but look for the family resemblances. Python and Ruby look identical here: no type, no semicolon. Java and C++ also match each other: both make you declare that `total` holds an integer (`int`) and end the line with a semicolon. JavaScript uses `let` to create a new variable, and Go uses its own `:=` symbol for the same job. Scratch replaces typing with blocks that snap together.

Notice how one line of Python turned into four lines of assembly. That is abstraction at work: **high-level** languages hide the hardware so you can write less, while **low-level** languages (assembly and machine language) give exact control but

make you spell out every step. Low-level code is also written for one specific kind of CPU. High-level code can run on many.

The translators

Something has to carry your code down the ladder. There are three main kinds of translator programs.

Assembler	Compiler	Interpreter
Turns assembly language into machine language, one instruction at a time.	Translates a whole program ahead of time and produces a file you can run later. C and C++ work this way.	Reads your program and carries it out as it goes, without producing a separate file first. Python works this way.

Some languages mix the two. Java is compiled into an in-between code called bytecode, which then runs on the Java Virtual Machine. That is how Java earned its slogan “write once, run anywhere”: any computer with the virtual machine can run the same bytecode.

Blocks or text?

Most of you have used Scratch, and you have been writing Python in CMU CS Academy. Both are high-level languages, and both end up as the same kind of machine instructions. The difference is in how you write them. Scratch blocks only snap together in ways that make sense, so you cannot make a typo or a syntax error, and you can focus on the logic. Python's text takes more care to type correctly, but gives you more precise control and scales up to much bigger programs.

ALL TOGETHER

What happens when you press Run

Let's follow one line of your Unit 3 code, `Circle(200, 200, 50)`, through the whole stack.

- 1 YOU**
You decide you want a circle in the middle of the canvas and type the line.
- 2 HARDWARE → OS → APPLICATION**
Each keypress travels from the keyboard (an input device) to the operating system, which passes it to your web browser, which passes it to CMU CS Academy.
- 3 PROGRAMMING LANGUAGE**
You press Run. Your Python is translated, piece by piece, into instructions the machine can carry out. In CMU CS Academy this happens inside your browser, which adds a translation layer of its own.
- 4 HARDWARE: MEMORY**
Those instructions, plus your program's data (the numbers 200, 200, and 50), are placed in RAM, side by side.
- 5 HARDWARE: CPU**
The CPU runs its fetch-decode-execute cycle millions of times, running the interpreter, CMU's graphics code, and the browser. The result is a drawing command: a circle, at this spot, this size, this color.

6

HARDWARE: GPU AND OUTPUT

The GPU works out which pixels fall inside the circle and colors them all at once. The operating system sends the finished image to your screen.

7

GATES AND TRANSISTORS

Underneath every one of those steps, logic gates are combining 1s and 0s, and transistors are switching on and off.

You wrote one line. Every layer below you did its job without you ever seeing it. That is the computing stack.

LOOKING AHEAD

Why are there so many languages?

By some counts, around 9,000 programming languages have been created, and 50 to 100 are in wide use today. If they all end up as machine instructions for the same kind of hardware, why not just use one? Because each language is built around a different set of **trade-offs**, and no language wins at everything.

TRADE-OFF	ONE END	THE OTHER END
Level	Close to the hardware: fast and precise, more work (C, assembly)	Far from the hardware: less code, the language handles the details (Python)
Translation	Compiled ahead of time (C, C++)	Interpreted as it runs (Python)
Purpose	General-purpose: good at many things (Python, Java)	Built for one job: statistics in R, databases in SQL
Form	Text-based (Python, Java)	Visual blocks (Scratch)

Languages also differ in **style**, or paradigm. Some spell out instructions step by step (imperative). Some bundle data and actions together into objects (object-oriented). Some build programs out of functions that avoid changing shared data (functional). And some things people call languages are not programming languages at all: HTML describes the structure of a web page and CSS describes its look, but neither one runs instructions.

★ CONNECTION

Next in class we sort programming languages into types. Every row of the table above is one way to sort them, so keep it in mind.

Check yourself

Play the Unit 0 review game: match the terms, sort the cards, and build the stack. Keep playing until you get a perfect score, then screenshot it and post it in the Teams assignment.

The game link is in the Teams assignment.